



МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Проектирование и разработка интерфейсов пользователя

Преподаватель

Коляда Никита Владимирович

Вебинар 2.3



1. Что такое ТЗ и с чего начать

Понимание задачи заказчика без страха и паники перед началом работы



2. Стек технологий под требования

Связка HTML5, CSS3 и JavaScript в проекте без использования бэкенда



3. Анатомия 6 страниц портала

Главная, каталог, бронирование, личный кабинет и страницы авторизации



4. Декомпозиция интерактивных фич

Слайдер, калькулятор стоимости, живой поиск, фильтрация и валидация



5. Архитектура файлов и тайминг

Организация структуры папок проекта и поминутный регламент на 120 минут



ПИРИП 2026 Задание на экзамен 1 семестр

ПМ08. Разработка веб-приложения.

Техническое задание на разработку интернет-портала бронирования офисных комнат «СмартОфис»

Описание задания: Разработать фронтенд-часть интернет-портала для бронирования офисных комнат в коворкинге. Пользователи могут просматривать каталог офисных комнат, оформлять бронирования (через клиентскую логику) и взаимодействовать с элементами интерфейса. Взаимодействие с сервером не требуется, все данные хранятся статично.

Время на выполнение: 2 часа

Технологии:

1. Frontend:
 - a. HTML5,
 - b. CSS3,
 - c. JavaScript (Vanilla JS).



Общие требования:

1. Использовать семантические теги (header, main, footer, nav, section).
2. Исходные изображения должны быть стилизованы (растянуты на весь блок без искажения, центрированы).
3. Корректное отображение на экранах с шириной от 1200px (адаптивность через Media Queries).
4. В формах необходимо реализовать клиентскую валидацию: требовать от пользователя заполнения всех полей (кроме необязательных).
5. Шапка и подвал сайта должны быть унифицированы и использоваться на всех страницах.
6. Подобрать цветовую гамму из 3-5 основных цветов.
7. Верстка: Flexbox или Grid.





Требования для каждой страницы:

1. Главная страница (index.html)

-
1. **Шапка:** Логотип (SVG), название «СмартОфис». Навигация: «Главная», «Каталог», «Войти» (статичные ссылки). Для имитации авторизации при клике на «Войти» перенаправлять на login.html.
 2. **Слайдер:** Блок с четырьмя изображениями (800x400px). Реализовать на Vanilla JS (автоматическая смена каждые 3 секунды + ручное управление стрелками).
 3. **Секция «Популярные офисные комнаты»:** Статичный блок из трех карточек (изображение 300x200px, название, оснащение, цена). Кнопка «Больше офисов» ведет на catalog.html.
 4. **Подвал:** Контактная информация (email, телефон).



2. Страница регистрации (register.html)

1. **Форма:** Поля (логин, пароль, подтверждение пароля, ФИО, email, телефон).
2. **Логика:** При нажатии кнопки «Зарегистрироваться» валидировать данные на стороне клиента. При успехе выводить всплывающее сообщение ('Пользователь зарегистрирован успешно!'). При ошибке (несовпадение паролей, неверный формат) — выделять поле красным цветом и выводить сообщение об ошибке.

3. Страница авторизации (login.html)

1. **Форма:** Поля «Логин» и «Пароль».
2. **Логика:** Кнопка «Войти» должна проверять корректность (например, жестко заданные данные в JS). При ошибке выводить текст «Неверный логин или пароль».

4. Страница каталога (catalog.html)

1. **Отображение:** Использовать массив JavaScript-объектов (mock-данные) для отрисовки карточек офисов.
2. **Фильтрация и сортировка:** Текстовое поле для поиска по названию и кнопки сортировки по цене (возрастание/убывание) должны фильтровать/сортировать массив и перерисовывать DOM-элементы.
3. **Бронирование:** Кнопка «Забронировать» открывает модальное окно или переводит на страницу booking.html.



5. Страница бронирования (booking.html)

1. **Форма:** Выбор комнаты (select), выбор даты (input type="date"), количество часов (number), комментарий.
2. **Логика:** Рассчитывать общую стоимость (JS) и выводить её в реальном времени. При нажатии «Забронировать» выводить всплывающее сообщение ('Бронирование создано! Номер заявки: №).

6. Страница «Мои бронирования» (my-bookings.html)

1. **Отображение:** Отобразить список бронирований из массива mock-данных. Если список пуст, выводить сообщение «У вас пока нет бронирований».

Структура проекта:

1. /index.html (главная страница)
2. /pages/register.html (страница регистрации)
3. /pages/login.html (страница авторизации)
4. /pages/catalog.html (страница каталога)
5. /pages/booking.html (страница бронирования)
6. /pages/my-bookings.html (страница личного кабинета)
7. /css/style.css (основные стили)
8. /js/main.js (основная логика и функции)
9. /js/data.js (массив mock-данных)
10. /img/ (папка для изображений)



Зачем разработчику разбирать ТЗ перед началом работы

МОДУЛЬ 1 Разбор ТЗ > Пошаговый пайплайн разработки

ПАЙПЛАЙН РЕШЕНИЯ ЗАДАЧИ:

1. ТЗ заказчика → 2. Декомпозиция → 3. Готовый портал

✗ Без декомпозиции ТЗ:

- Паника перед 6 страницами
- Забыли валидацию и формы
- Переделка верстки в конце
- Нехватка времени на экзамене

✓ С разбором по шагам:

- Четкий список всех страниц
- Понятные роли HTML, CSS, JS
- Спокойствие и контроль кода
- 100 баллов за 120 минут

Ключевые рамки: 120 минут экзамена Объем: 6 связанных страниц Сервер: 0% (только клиент)

Зачем разбирать ТЗ до первой строчки кода

- 1. Карта пути проекта:**
ТЗ — это не сухой список ограничений, а пошаговый план, защищающий от бесконечных переделок и хаоса в коде.
- 2. Точные критерии экзамена:**
Эксперты оценивают проект строго по пунктам ТЗ: наличие семантики, валидации, адаптивности и структуры папок.
- 3. Визуальный образ результата:**
Пока вы не представили готовый сайт в голове и на схеме, писать код бесполезно — легко свернуть не туда.
- 4. Спокойствие и уверенность:**
Когда сложное задание разбито на 10 простых шагов, страх исчезает — вы решаете понятные микрозадачи.



Чек-лист первичного аудита технического задания

ЧЕК-ЛИСТ Анализ требований > 4 главных вопроса

1. ЧТО МЫ СОЗДАЕМ?
Интернет-портал бронирования офисных комнат «СмартОфис» с каталогом, слайдером, калькулятором брони и ЛК.

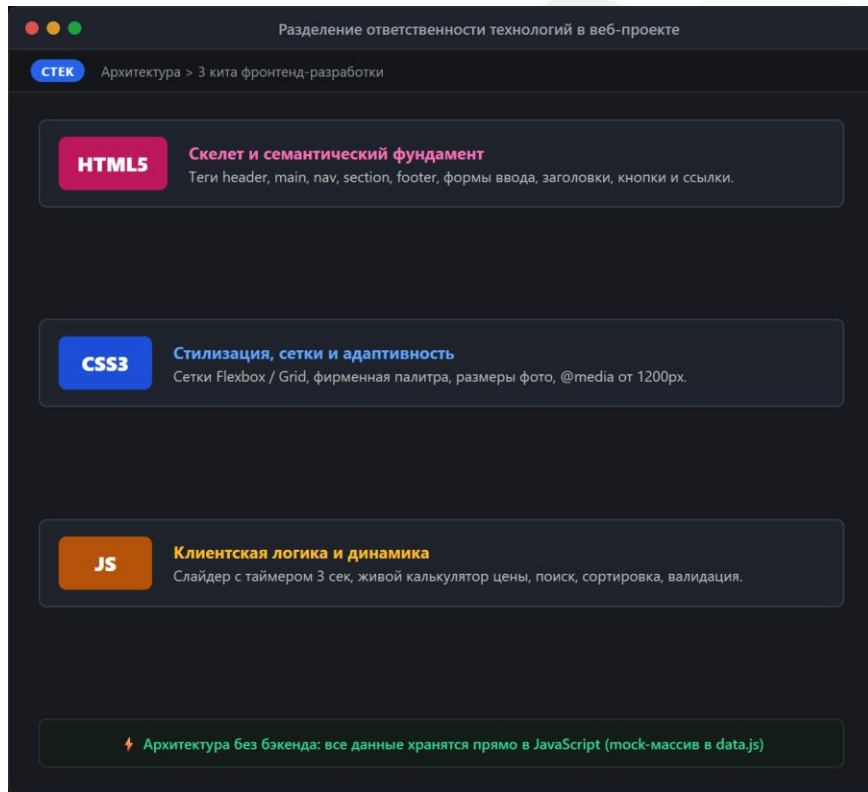
2. ДЛЯ КОГО ЭТОТ СЕРВИС?
Для клиентов и резидентов коворкинга, которым нужно быстро найти офис, узнать цену и оформить бронь.

3. КАКИЕ ТЕХНОЛОГИИ РАЗРЕШЕНЫ?
Строго чистый фронтенд-стек: HTML5, CSS3 и Vanilla JavaScript. Запрещено использовать React, Vue или внешние библиотеки!

4. СКОЛЬКО ВРЕМЕНИ НА ВЫПОЛНЕНИЕ?
Ровно 2 часа (120 минут). Каждая минута должна быть распланирована по этапам заранее.

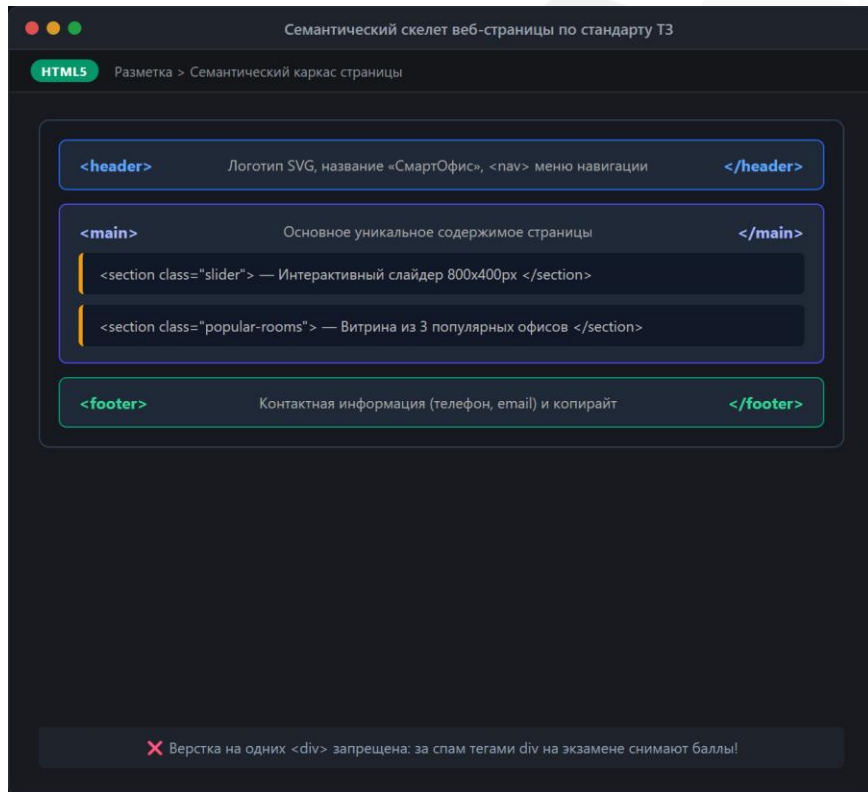
Чек-лист первичного аудита ТЗ

- 1. ЧТО мы создаем?:**
Интернет-портал бронирования офисных комнат «СмартОфис» с каталогом, бронью и личным кабинетом.
- 2. ДЛЯ КОГО этот сервис?:**
Для клиентов коворкинга, которым нужно быстро найти офис, рассчитать цену за часы и оформить бронь.
- 3. КАКИЕ технологии разрешены?:**
Строго чистый стек без фреймворков: HTML5, CSS3 и Vanilla JavaScript. Никаких сторонних библиотек!
- 4. СКОЛЬКО времени на работу?:**
Ровно 2 часа (120 минут). Каждая минута на счету, поэтому план работы должен быть поминутным.



Разделение ответственности технологий

- **1. HTML5 — скелет и структура:**
Отвечает за все элементы: кнопки, поля ввода, заголовки, списки и смысловые блоки страниц.
- **2. CSS3 — красота и адаптивность:**
Задаёт цвета (3–5 оттенков), сетки Flexbox/Grid, размеры картинок и адаптив под экраны от 1200px.
- **3. Vanilla JS — поведение и логика:**
Оживляет интерфейс: автослайдер, фильтрация каталога, валидация полей и мгновенный расчет цены.
- **4. Без серверного бэкенда:**
По ТЗ сервер не требуется: данные хранятся в mock-массивах в памяти браузера, что сильно упрощает задачу.



Отказ от бессмысленных div-оберток

1. Понятный код с первого взгляда:

Семантические теги моментально показывают назначение блока: шапка, навигация, секция или подвал.

2. Требование профессиональных стандартов:

В экзаменационном чек-листе за разметку на одних `<div>` ставят штрафные баллы за грязь в коде.

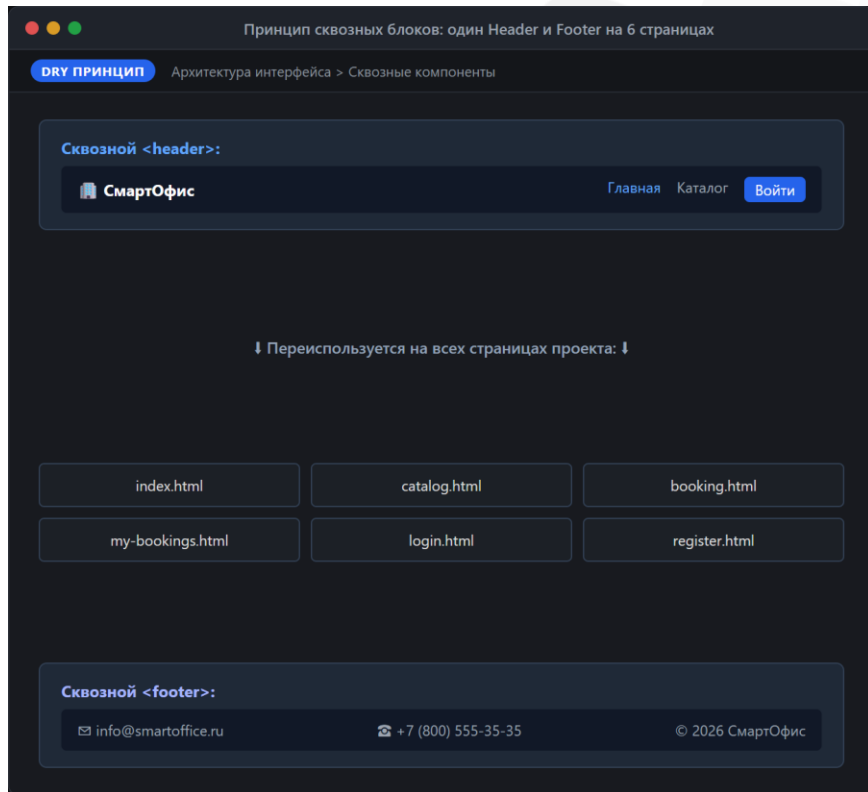
3. Доступность (Accessibility):

Экранные дикторы для людей с ОВЗ и поисковые роботы ориентируются именно по тегам `<header>`, `<nav>` и `<main>`.

4. Подготовка к Вебинару 2:

Уже на следующем занятии мы начнем верстку именно с создания этого чистого семантического каркаса.

Зачем в T3 строго требуют теги header, main, nav, section и footer?



Принцип DRY: верстаем один раз

1. Единый Header на 6 страницах:

Логотип «СмартОфис», ссылки «Главная», «Каталог» и «Войти» выглядят и работают одинаково везде.

2. Единый Footer с контактами:

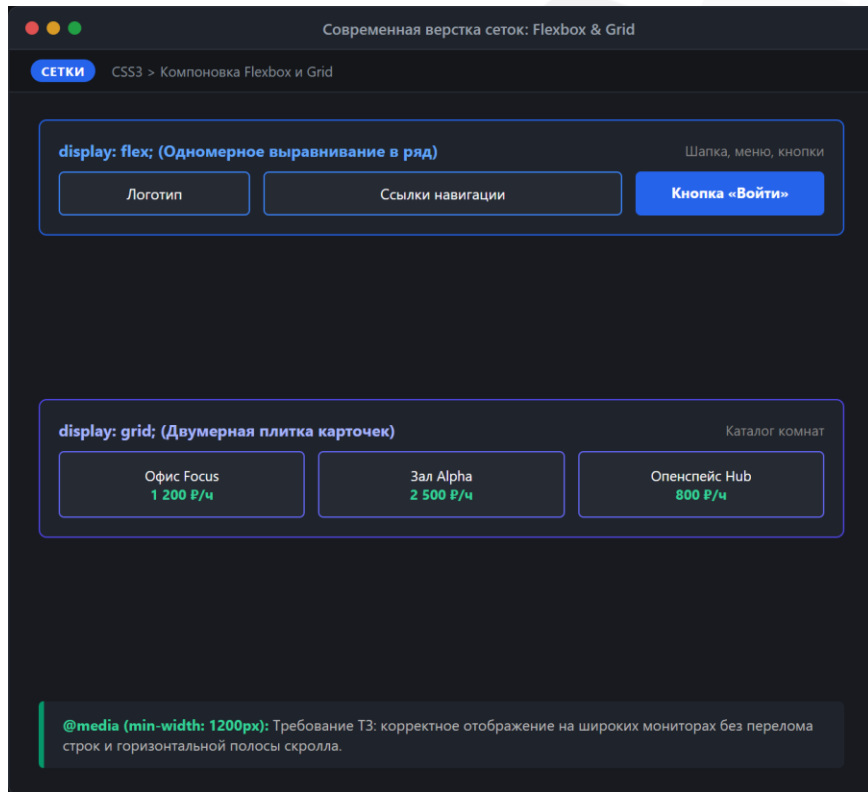
Телефон и email размещаются внизу каждой страницы в одинаковом оформлении и координатах.

3. Экономия времени на экзамене:

Вы сверстали шапку и подвал один раз на index.html, а затем просто копируете их в остальные 5 страниц.

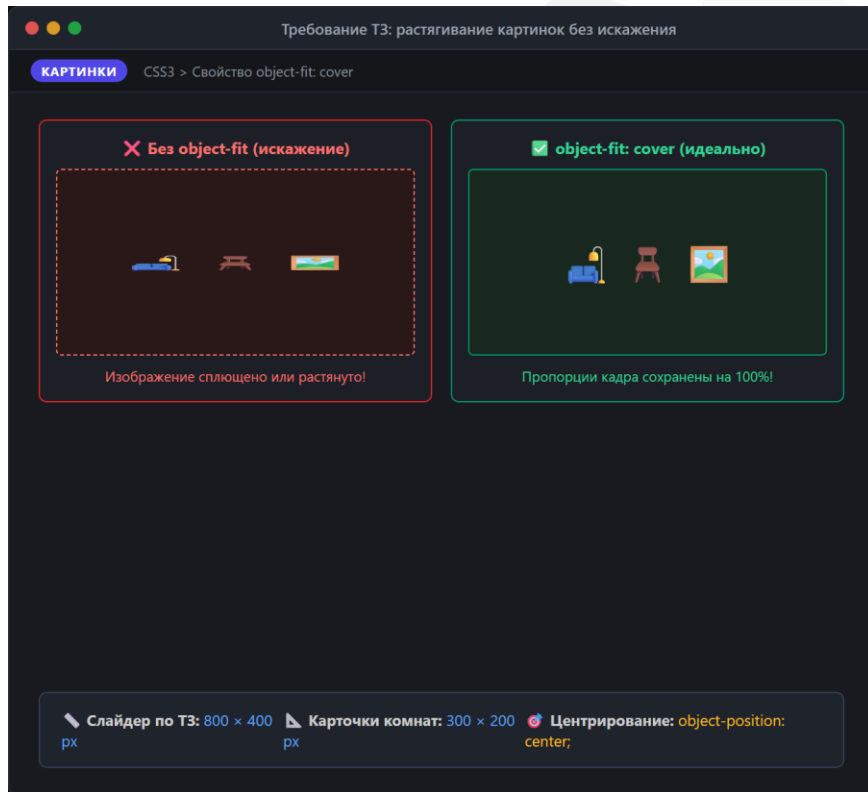
4. Удобство пользователя:

Посетитель сайта не теряется при переходах между разделами — навигация всегда под рукой.



Современные механизмы компоновки

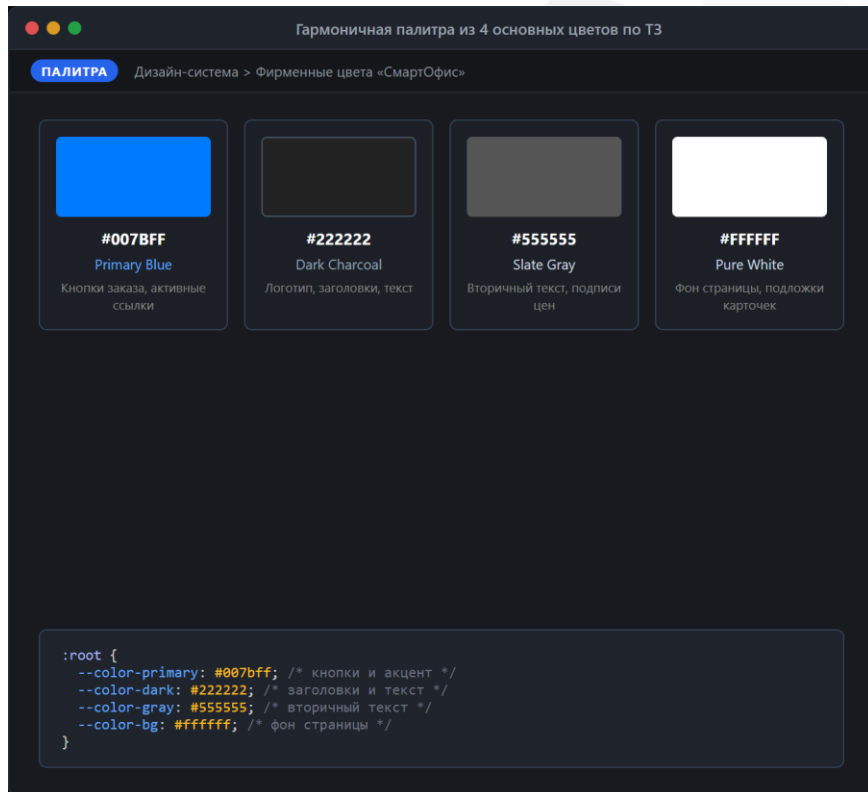
- 1. **Flexbox для одномерных рядов:**
Идеально подходит для шапки (логотип слева, меню справа) и кнопок сортировки в одну строчку.
- 2. **Grid для двумерных каталогов:**
Позволяет строить ровные плитки карточек офисных комнат с автоматическим расчетом колонок.
- 3. **Адаптивность от 1200px:**
Медиа-запросы @media гарантируют аккуратное отображение карточек без горизонтального скролла.
- 4. **Карточки равной высоты:**
Flexbox и Grid автоматически выравнивают карточки комнат по высоте, даже если описания разной длины.



Работа с пропорциями картинок

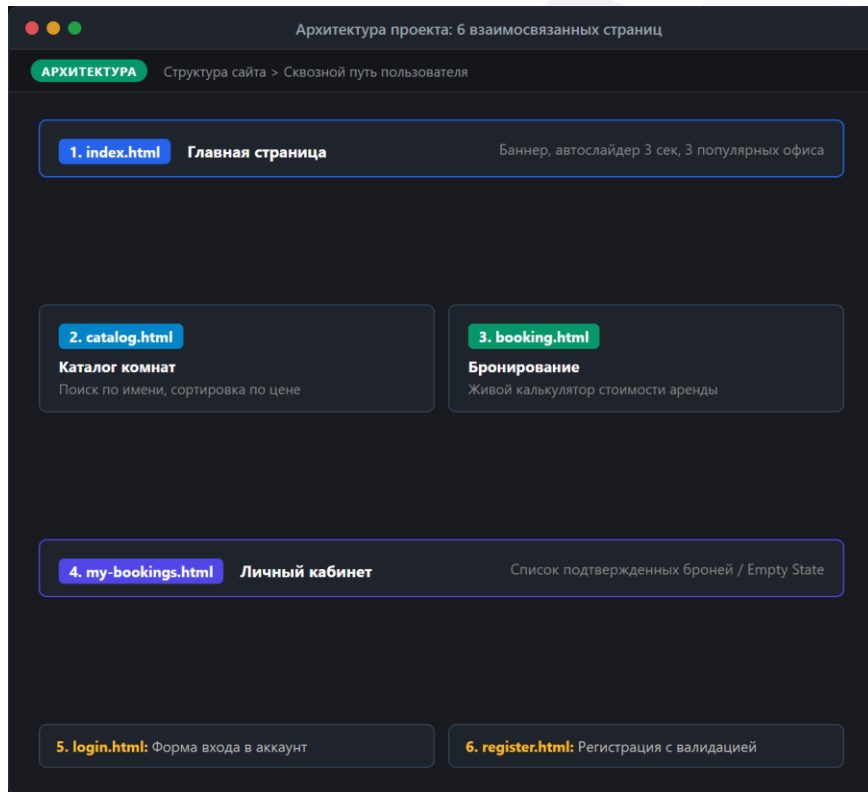
- 1. **Требование Т3: без искажений:**
Фотографии комнат имеют разные пропорции. Растягивать их обычным width/height 100% нельзя — они сплющатся.
- 2. **Спасительное свойство object-fit:**
Значение object-fit: cover сохраняет идеальные пропорции кадра, мягко обрезая лишние края.
- 3. **Четкие размеры из Т3:**
В Т3 заданы конкретные габариты: слайдер 800×400px, карточки комнат 300×200px. Верстаем строго по пикселям.
- 4. **Центрирование object-position:**
Параметр object-position: center удерживает фокус на интерьере комнаты независимо от разрешения.

Как растянуть фото комнат на весь блок без искажений по требованию Т3?



Системный подбор цветов для сервиса

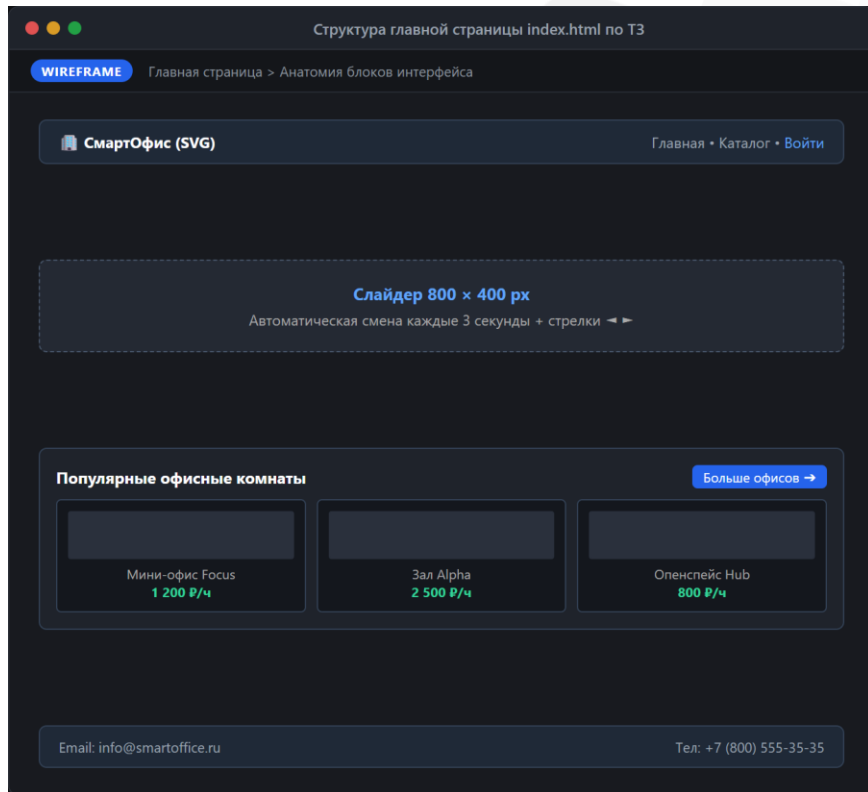
- 1. Правило 3–5 цветов по Т3:**
Не превращайте сайт в новогоднюю елку. Достаточно 4 цветов: фон, текст, нейтральный и акцент.
- 2. Основной синий (#007BFF):**
Цвет акцента и кнопок целевого действия: «Забронировать», «Войти», ссылки навигации.
- 3. Темный угольный (#222222):**
Базовый цвет для контрастных заголовков, основного текста и элементов интерфейса.
- 4. Чистый белый и серый (FFFFFF, #555555):**
Фон страницы, подложки карточек и приглушенный серый текст для комфорта глаз.



Сквозной пользовательский сценарий

- 1. **Точка входа: Главная (index.html):**
Визитная карточка сервиса: баннер, слайдер интерьеров и блок 3 популярных офисов.
- 2. **Каталог и Бронирование:**
Пользователь переходит в catalog.html, выбирает офис и оформляет заявку на booking.html.
- 3. **Личный кабинет (my-bookings.html):**
Страница со списком всех подтвержденных броней пользователя или заглушкой «Нет броней».
- 4. **Авторизация и Регистрация:**
Служебные страницы login.html и register.html для входа и проверки клиентской валидации.

Из каких страниц состоит портал «СмартОфис» и как они связаны между собой?



Анатомия главной страницы

- 1. **Шапка сайта (Header):**
Логотип в формате SVG, текстовое имя «СмартОфис» и меню: «Главная», «Каталог», «Войти».
- 2. **Слайдер 800x400px:**
Центральный интерактивный блок с четырьмя качественными фотографиями офисных помещений.
- 3. **Секция «Популярные офисы»:**
Статичный блок из 3 карточек (картинка 300×200, название, оснащение, цена) и кнопка «Больше офисов».
- 4. **Подвал (Footer):**
Нижняя плашка с контактным email и телефоном службы поддержки коворкинга.



Механика автослайдера на чистом JavaScript (Vanilla JS)

СЛАЙДЕР JavaScript > Автосмена каждые 3 секунды

Цикл смены слайдов:

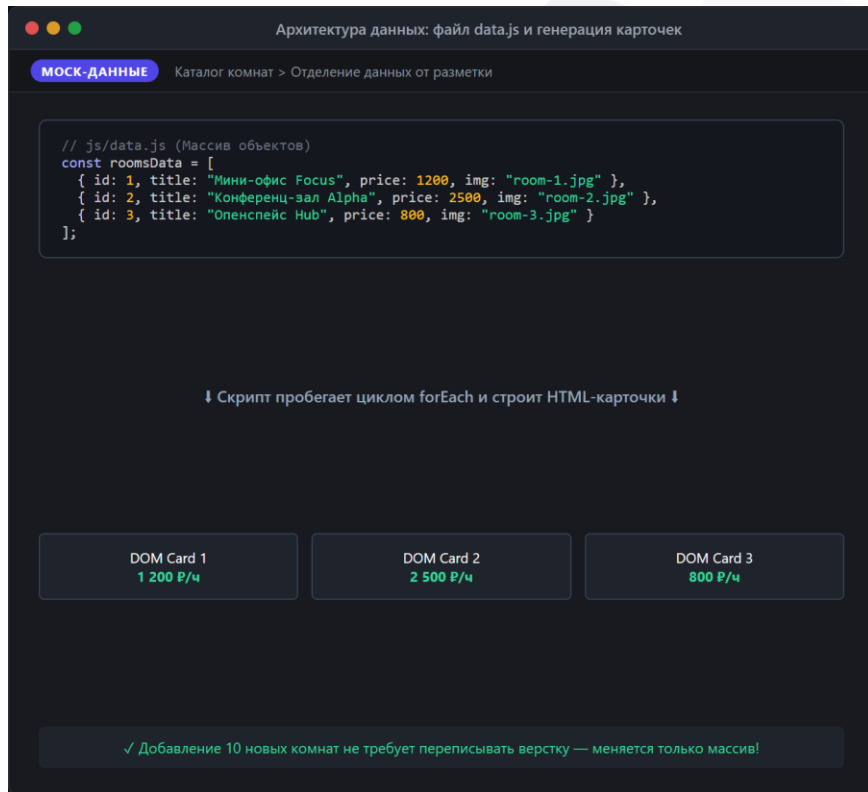
Слайд 1 → 3 сек → Слайд 2 → 3 сек → Слайд 3 → 3 сек → Слайд 4

```
let currentIndex = 0;
function nextSlide() {
  currentIndex = (currentIndex + 1) % 4;
  showSlide(currentIndex);
}
setInterval(nextSlide, 3000); // каждые 3 секунды
```

Ручные стрелки: prevBtn (←) и nextBtn (→) ✓ Изучим на Вебинаре 7

Алгоритм работы слайдера на JS

- **1. Хранилище картинок и индекс:**
В JS заводим массив путей к 4 фото и переменную `currentSlide = 0` для текущего кадра.
- **2. Автопрокрутка через `setInterval`:**
Функция `setInterval(nextSlide, 3000)` запускает смену слайда каждые 3000 миллисекунд.
- **3. Ручное управление стрелками:**
Кнопки «Влево» и «Вправо» сбрасывают таймер и листают слайды вперед или назад по кругу.
- **4. Изучим на Вебинаре 7:**
На 7 вебинаре мы подробно разберем таймеры и напомним этот слайдер всего за 25 строчек кода.



Архитектура данных без бэкенда

1. Файл data.js как мини-база:

Вместо сложной базы данных создаем JS-массив объектов с полями: id, title, price, image, features.

2. Генерация через цикл forEach:

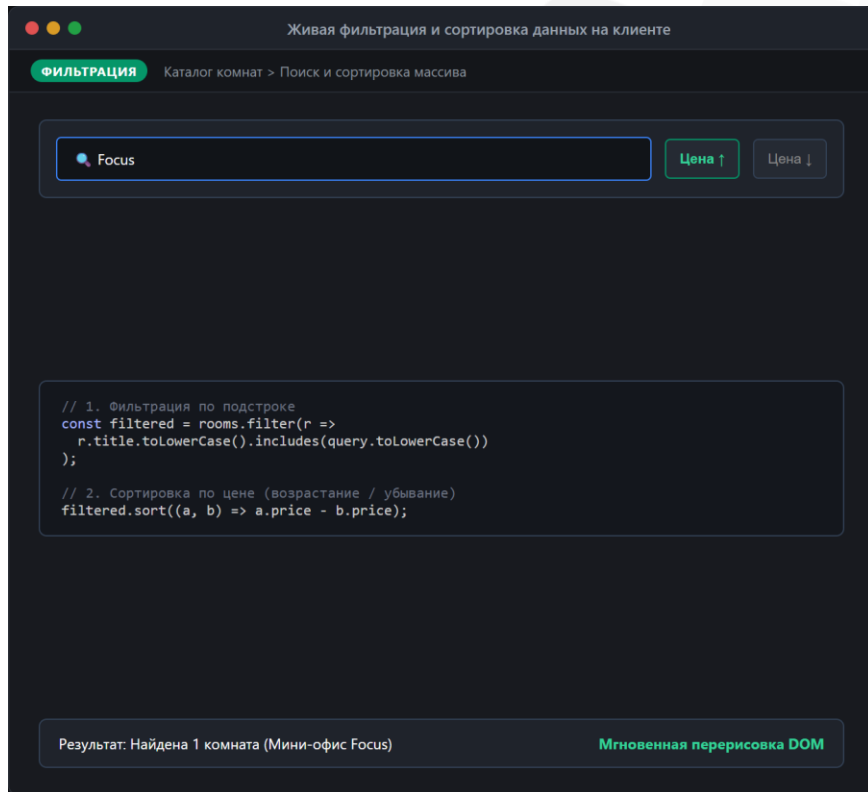
Скрипт пробегает по массиву и динамически собирает HTML-разметку для каждой комнаты.

3. Разделение данных и верстки:

Если заказчик добавит 10 новых комнат, верстку переписывать не придется — достаточно обновить массив.

4. Изучим на Вебинаре 5:

На 5 вебинаре мы освоим генерацию DOM-элементов и подключим моск-данные к каталогу.



Обработка данных на клиенте

1. Поиск по названию комнаты:

Событие input отслеживает ввод пользователя и фильтрует массив методом `array.filter()`.

2. Сортировка дешевле / дороже:

Кнопки сортировки применяют метод `array.sort()` по числу `price` и сразу вызывают перерисовку.

3. Мгновенный отклик без перезагрузки:

Так как все данные уже в браузере, фильтрация происходит молниеносно за 1 миллисекунду.

4. Изучим на Вебинаре 8:

На 8 вебинаре мы по шагам соберем связку: поле ввода → фильтр → сортировка → перерисовка карточек.



Живой калькулятор стоимости аренды на booking.html

КАЛЬКУЛЯТОР Бронирование > Формула и модальное окно заявки

Выбор комнаты (select): Конференц-зал Alpha (2 500 Р/ч)

Количество часов (input type="number"): 4 часа

ИТОГО К ОПЛАТЕ (В РЕАЛЬНОМ ВРЕМЕНИ):

10 000 Р

Формула: $2\,500\text{ Р} \times 4\text{ часа} = 10\,000\text{ Р}$

Всплывающее уведомление по ТЗ:
«Бронирование создано! Номер заявки: №74921»

Формула и логика калькулятора

- 1. **Поля формы по ТЗ:**
Выпадающий список комнат (select), выбор даты (input date), количество часов (input number).
- 2. **Расчет в реальном времени:**
Событие change/input перемножает цену выбранной комнаты на число часов: Итого = Цена x Часы.
- 3. **Генерация номера заявки:**
При клике «Забронировать» JS генерирует случайный номер ($\text{№} + \text{Math.floor}$) для всплывающего окна.
- 4. **Защита от отрицательных чисел:**
Атрибут min='1' и проверка в JS запрещают бронировать комнату на 0 или отрицательное время.



Клиентская валидация форм на стороне браузера

ВАЛИДАЦИЯ Формы > Проверка обязательных полей и паролей

Подтверждение пароля: Пароли не совпадают!

..... (подсветка поля красным цветом)

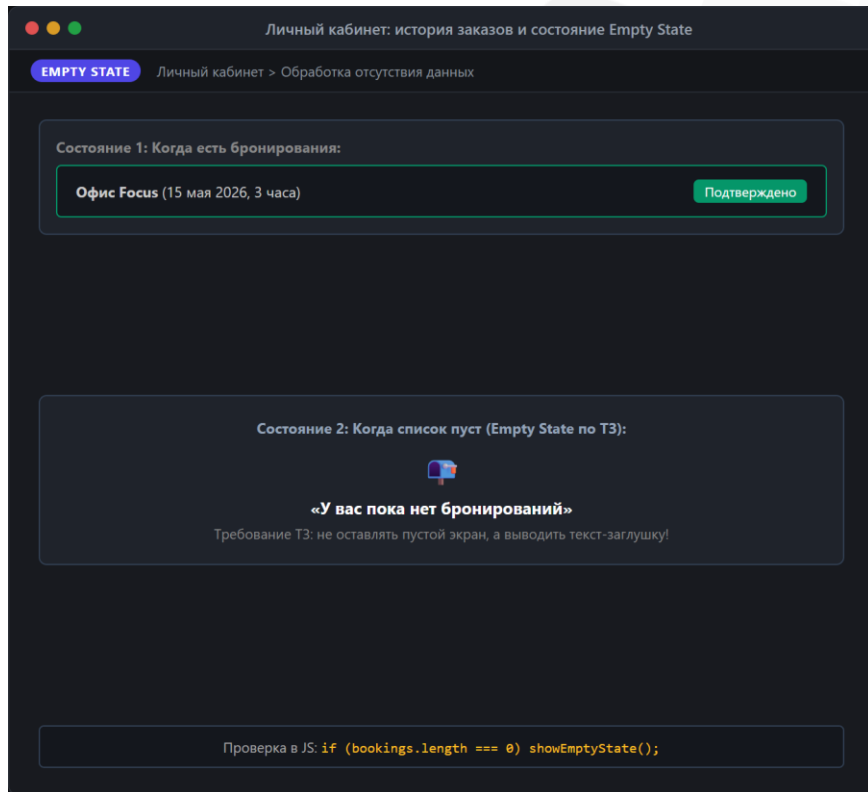
Email пользователя: Корректный формат ✓

ivanov@smartoffice.ru (подсветка зеленым цветом)

При успехе: «Пользователь зарегистрирован успешно!» Вебинар 6

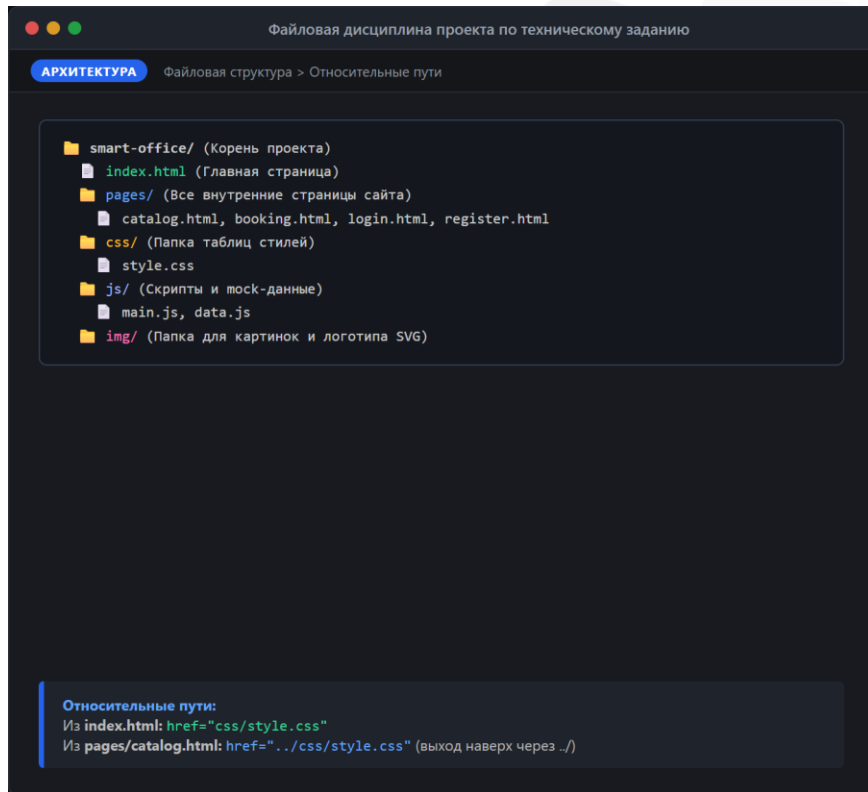
Защита форм регистрации и входа

- 1. **Требование ТЗ к полям:**
Запрещено отправлять пустые поля. Логин, email, телефон и пароли должны быть заполнены.
- 2. **Проверка совпадения паролей:**
Если пароль и подтверждение не совпадают — красим рамку инпута в красный цвет (#EF4444).
- 3. **Всплывающее сообщение об успехе:**
При корректных данных выводим: «Пользователь зарегистрирован успешно!» через alert/modal.
- 4. **Изучим на Вебинаре 6:**
На 6 вебинаре мы детально разберем регулярные выражения, валидацию форм и стили ошибок.



Обработка двух состояний экрана

- 1. **Если бронирования есть:**
Отрисовываем аккуратный список карточек с названием офиса, датой, часами и статусом «Подтверждено».
- 2. **Если список пуст (length === 0):**
По ТЗ обязательно выводим сообщение-заглушку: «У вас пока нет бронирований».
- 3. **Профессиональный UX:**
Интерфейс никогда не должен оставаться глухим белым экраном при отсутствии данных.
- 4. **Изучим на Вебинаре 9:**
На финальном 9 вебинаре мы свяжем бронирование с личным кабинетом и соберем проект целиком.



Файловая дисциплина по ТЗ

- 1. **Главная в корне: index.html:**
Файл index.html обязан лежать строго в корне проекта — именно его первым открывает сервер.
- 2. **Подстраницы в папке /pages/:**
Файлы catalog.html, booking.html, login.html, register.html, my-bookings.html хранятся в /pages/.
- 3. **Папки ресурсов: css/, js/, img/:**
Стили в /css/style.css, скрипты в /js/main.js и /js/data.js, все фото комнат — в /img/.
- 4. **Относительные пути ../:**
Помните: из папки /pages/ путь к стилям пишется через выход наверх: href='../css/style.css'.



Регламент 120 минут экзамена: поминутный тайм-менеджмент	
ТАЙМИНГ 120 МИН Экзамен > Поминутный план успешной сдачи	
1. Каркас и сквозные блоки (0–30 мин) Структура папок, style.css, семантический Header и Footer.	30 мин
2. Верстка всех 6 страниц (30–70 мин) Главная, каталог комнат, формы бронирования и авторизации.	40 мин
3. JavaScript-интерактив (70–100 мин) Слайдер 3 сек, живой калькулятор цен, фильтры каталога, валидация.	30 мин
4. Тестирование и полировка (100–120 мин) Чек-лист ТЗ, проверка консоли, адаптивность от 1200px, сдача работы!	20 мин

Тайм-менеджмент на экзамене

- 1. **Минуты 0–30: Каркас и стили:**
Создаем структуру папок, верстаем семантическую шапку, подвал и подключаем style.css.
- 2. **Минуты 30–70: Верстка 6 страниц:**
Копируем сквозные блоки и верстаем разметку главной, каталога, бронирования и ЛК.
- 3. **Минуты 70–100: Логика JavaScript:**
Подключаем data.js, выводим карточки, настраиваем слайдер, калькулятор и валидацию.
- 4. **Минуты 100–120: Тестирование:**
Проверяем все ссылки, формы, консоль браузера на ошибки и адаптивность от 1200px.



Каталог офисных пространств

Выберите подходящее помещение для индивидуальной работы или командных встреч

Поиск по названию офиса...

Цена: по возрастанию ↑

Цена: по убыванию ↓



Мини-офис Focus

Wi-Fi 500 Мбит/с
4K Монитор
Эргономичное кресло
Климат-контроль



Конференц-зал Alpha

Проектор 4K
Спикерфон
Флипчарт
Аудиосистема



Опенспейс Hub

Личный стол
Wi-Fi 500 Мбит/с
Розетки 220V и USB
Кофе-поинт



МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

ДО ВСТРЕЧИ!

УВИДИМСЯ НА СЛЕДУЮЩЕМ ЗАНЯТИИ